CSE331 - Assignment #2

Seong-Ji Yang (20211181)
UNIST
South Korea
yangsj@unist.ac.kr

1 INTRODUCTION

The Traveling Salesman Problem (TSP) seeks the shortest possible route that visits each city exactly once and returns to the starting point. As an NP-hard problem, finding the exact solution for large instances is computationally infeasible, so This document approaches TSP problem with practical approaches relying on approximation and heuristic algorithms, with source codes and data sets which are publicly available in **here**[4].

2 PROBLEM STATEMENT

This report focuses on two established algorithms: the MST-based 2-approximation algorithm, which guarantees a tour no more than twice the optimal length, and the Held-Karp algorithm, a dynamic programming method that finds the exact solution but is only practical for small datasets due to its exponential time complexity. Additionally, the Christofides heuristic[3], which achieves a 1.5-approximation ratio, will be used to compare the results.

Finally, a novel algorithm developed for this project is introduced and evaluated along with existing methods, with all approaches compared in terms of theoretical complexity, empirical runtime, and approximation quality on standard benchmark datasets.

3 EXISTING ALGORITHMS

This section analyzes two algorithms for solving the Traveling Salesman Problem: the MST-based 2-approximation algorithm and the Held-Karp dynamic programming algorithm. Both are implemented to handle complete graphs where the cost of travel between cities is specified by the Euclidean distance rounded to the nearest whole number using the formula round($\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$).

3.1 MST-Based 2-Approximation Algorithm

The MST-based 2-approximation algorithm(MST2) provides polynomial time solution to TSP with a guaranteed approximation ratio[1]. The algorithm consists of four main phases: constructing a minimum spanning tree, duplicating edges, finding an Eulerian circuit, and performing shortcutting.

3.1.1 Algorithm Structure and Implementation. **Phase 1: Minimum Spanning Tree Construction.** The implementation uses memory-optimized Prim's algorithm. Instead of maintaining a complete adjacency matrix of size $O(n^2)$, the algorithm uses a priority vector of tuples $\langle$distance, from, to$\rangle$. For each iteration i, the algorithm targets $n-i$ candidate edges, with n as the number of vertices.

The time complexity of this phase is $O(n^2)$. At iteration i, the algorithm updates distances for $n-i$ remaining vertices and finds the minimum among them, requiring $O(n-i)$ operations. The total time complexity is:

$$T_{\text{Prim}} = \sum_{i=1}^{n-1} O(n-i) = O\left(\sum_{i=1}^{n-1} i\right) = O\left(\frac{n(n-1)}{2}\right) = O(n^2) \quad (1)$$

The space complexity is $O(n)$ due to the optimized priority vector representation, significantly better than the standard $O(n^2)$ adjacency matrix approach.

Phase 2: Edge Duplication. All edges in the MST are duplicated to make every vertex have even degree. This phase requires $O(n)$ time since the MST contains exactly $n-1$ edges, and each edge duplication involves constant-time list operations. The space complexity is $O(n)$ for storing the adjacency lists.

Phase 3: Eulerian Circuit Construction. The algorithm constructs an Eulerian circuit using DFS approach. From an arbitrary vertex, it performs DFS, removing edges as they are traversed. If a vertex has no remaining unvisited edges, it is added to the circuit.

The time complexity is $O(n)$ since each of the $2(n-1)$ edges in the duplicated graph is visited exactly once. The space complexity is $O(n)$ for the stack and result storage.

Phase 4: Shortcutting. The final phase converts the Eulerian circuit into a Hamiltonian cycle by visiting each vertex exactly once. It tracks visited vertices and skips already-visited vertices, connecting directly to the next unvisited vertex.

For a circuit of length $2n-2$ (visiting n distinct vertices), the algorithm processes each position once, requiring $O(n)$ time. The space complexity is $O(n)$ to check the visited array.

3.1.2 Overall Complexity Analysis. The total time complexity is dominated by the MST construction phase:

$$\begin{aligned} T_{\text{total}} &= T_{\text{Prim}} + T_{\text{duplicate}} + T_{\text{Euler}} + T_{\text{shortcut}} \\ &= O(n^2) + O(n) + O(n) + O(n) = O(n^2) \end{aligned} \quad (2)$$

The total space complexity is:

$$S_{\text{total}} = max(S_{\text{Prim}}, S_{\text{adjacency}}, S_{\text{Euler}}, S_{\text{shortcut}}) = O(n) \quad (3)$$

This represents a significant space improvement over standard implementations that use $O(n^2)$ adjacency matrices.

3.1.3 Approximation Analysis. Let C^* denote the optimal TSP tour cost and C_{MST} denote the cost of the minimum spanning tree. The approximation ratio can be seen with the following relationship:

$$C_{\text{MST}} \leq C^* \quad (4)$$

This holds because removing any edge from the optimal tour creates a spanning tree, and the MST is the minimum such tree.

The Eulerian circuit has cost $2 \cdot C_{\text{MST}} \leq 2 \cdot C^*$. During shortcutting, the triangle inequality ensures that the direct distance between any two vertices is at most the sum of distances through intermediate vertices. Therefore:

$$C_{\text{approx}} \leq 2 \cdot C_{\text{MST}} \leq 2 \cdot C^* \quad (5)$$

This equation shows the approximation ratio with tight bound:

$$r = \frac{C_{\text{approx}}}{C^*} \leq 2 \tag{6}$$

3.2 Held-Karp Dynamic Programming Algorithm

The Held-Karp algorithm provides exact solution to the TSP with dynamic programming[2]. It used little optimization to reduce space complexity compared to the standard formulation.

3.2.1 Algorithm Structure and Implementation. **State Representation and Optimization.** The implementation optimizes space with the fact that vertex i must be in the subset represented by *mask* for state $dp[i][mask]$. Therefore, the algorithm stores $dp[i][mask-2^i]$, reducing the space by eliminating redundant storage where vertex i is not in the mask, requiring:

$$\text{Space for vertex } i = 2^n - 2^i \text{ states} \tag{7}$$

The total space complexity becomes:

$$\begin{aligned} S_{\text{total}} &= \sum_{i=0}^{n-1} (2^n - 2^i) \\ &= n \cdot 2^n - \sum_{i=0}^{n-1} 2^i \\ &= n \cdot 2^n - (2^n - 1) \\ &= (n-1) \cdot 2^n + 1 \end{aligned} \tag{8}$$

This was reduced from the standard space $n \cdot 2^n$.

Dynamic Programming Recurrence. The algorithm implements the recurrence relation:

$$\text{dp}[i][\text{mask}] = \min_{j \in \text{mask},\, j \neq i} (\text{dp}[j][\text{mask} \setminus \{i\}] + d(j,i)) \tag{9}$$

where *mask* represents the bitmask of intermediate vertices visited before reaching vertex i.

Base Case Initialization. The algorithm initializes:

$$\begin{aligned} dp[0][0] &= 0 \\ dp[i][0] &= d(0,i) \text{ for } i = 1, 2, \ldots, n-1 \end{aligned} \tag{10}$$

This represents paths from the starting vertex to other vertices.

3.2.2 Complexity Analysis. **Time Complexity.** It iterates all bitmasks from 1 to $2^n - 1$. For each *mask* with k bits set, it considers each vertex i in the mask and each possible previous vertex j.

The total number of operations is:

$$\begin{aligned} T_{\text{total}} &= \sum_{k=2}^{n} \binom{n}{k} \cdot k \cdot (k-1) \\ &= \sum_{k=2}^{n} \frac{n!}{(n-k)!} \cdot (k-1) \end{aligned} \tag{11}$$

Using the binomial theorem and asymptotic analysis:

$$T_{\text{total}} = O(n^2 \cdot 2^n) \tag{12}$$

Space Complexity. With the optimization described above:

$$S_{\text{total}} = O((n-1) \cdot 2^n) = O(n \cdot 2^n) \tag{13}$$

3.2.3 Exactness Analysis. The Held-Karp algorithm provides the optimal solution to the TSP. The approximation ratio is:

$$r = \frac{C_{\text{Held-Karp}}}{C^*} = 1 \tag{14}$$

The algorithm explores all possible subsets of vertices and all possible orderings within each subset through dynamic programming. The optimal substructure property ensures that:

If the optimal tour visits vertices in subset S and ends at vertex i, then the subpath visiting $S \setminus \{i\}$ and ending at any vertex $j \in S \setminus \{i\}$ must also be optimal.

This property guarantees that the dynamic programming recurrence captures all possible optimal solutions, making the algorithm exact rather than approximate. The trade-off for exactness is exponential complexity, with memory requirements growing as $n \cdot 2^n$ and computation time growing as $n^2 \cdot 2^n$. For practical instances, the algorithm becomes computationally infeasible when $n > 25$ due to both time and space constraints.

4 PROPOSED ALGORITHM(S)

4.1 Layered Convex Hull Greedy Merge Heuristic

The proposed algorithm addresses the fundamental issue that crossing paths in TSP tours significantly degrades solution quality. From problem solving experience with CCW (Counter-ClockWise) operations for line segment intersection detection, this approach employs convex hull decomposition to minimize path crossings while maintaining computational efficiency.

The algorithm motivation stems from two key observations: first, that eliminating all crossing segments using CCW would be irrational, which resulted in convex hull layers using CCW. The solution constructs nested convex hull layers, builds partial TSP paths within inner layers, and gradually merges outer layer nodes to ensure minimal crossing.

To address performance degradation in concave configurations (such as Pacman-shaped distributions where the "mouth" corresponds to inner layers), the algorithm uses greedy strategy to choose insertion points with minimal TSP path length increase among multiple candidates.

4.1.1 Algorithm Structure and Implementation. **Phase 1: Layer Decomposition.** The algorithm uses Andrew's monotone chain algorithm to iteratively compute convex hulls. For each iteration, the algorithm constructs upper and lower hulls using CCW, and removes hull points from the active set. This process continues until fewer than three points remain, creating a layer structure.

Phase 2: Greedy Layer Merging. For each point in a new layer, the algorithm maintains a candidate set of the $N = 10$ nearest points from the current TSP path. The merging cost between point p and path segment (u, v) is calculated as the difference between new paths and original path length. The algorithm distinguishes between adjacent insertions (where candidates are consecutive in the path) and non-adjacent insertions (requiring path reversal).

Phase 3: Path Optimization. The final TSP path is computed by connecting all points in the merged sequence, with distance calculations using Euclidean distance rounded to the nearest integer.

Below are pseudocodes for implementing the algorithm:

Algorithm 1 Layered Convex Hull TSP Algorithm

```
function LayeredConvexHullTSP(points)
    layers ← DivideLayers(points)
    if layers is empty then
        return 0
    tsp_path ← layers[|layers| − 1]
    Remove last element from layers
    for i ← |layers| − 1 down to 0 do
        MergeLayer(tsp_path, layers[i])
    return CalculateCycleLength(tsp_path)
```

Algorithm 2 Convex Hull Layer Decomposition

```
function DivideLayers(points)
    layers ← ∅
    while points ≠ ∅ do
        hull ← ConvexHull(points)
        if |hull| < 3 then
            if hull ≠ ∅ then
                Add hull to layers
            break
        Add hull to layers
        Remove hull points from points
    return layers
```

Algorithm 3 Greedy Layer Merging with Candidate Selection

```
function MergeLayer(merged, new_layer)
    N ← 10
    while new_layer ≠ ∅ do
        Find point p with minimum distance to merged
        Remove p from new_layer
        candidates ← top N nearest points in merged to p
        min_cost ← ∞
        for each pair (a, b) in candidates do
            cost ← CalculateInsertionCost(p, a, b, merged)
            if cost < min_cost then
                min_cost ← cost
                best_position ← (a, b)
        Insert p at best_position in merged
```

4.1.2 Complexity Analysis. **Time Complexity.** Let n_i denote the number of points remaining before computing the i-th convex hull layer. For m layers with h_i points in layer i, the convex hull computation requires:

$$T_{\text{decomposition}} = \sum_{i=1}^{m} O(n_i \log n_i) \tag{15}$$

In the worst-case scenario where each layer contains only 3 points ($h_i = 3$), the number of layers becomes $m = \lfloor n/3 \rfloor$. The remaining points after each iteration follow $n_{i+1} = n_i - 3$, leading to:

$$\begin{aligned} T_{\text{decomposition}} &= \sum_{k=0}^{\lfloor n/3 \rfloor} O((n-3k)\log(n-3k)) \\ &= \sum_{k=1}^{n} O(k \log k) \cdot \frac{1}{3} \quad \text{(substitution } k = n - 3i\text{)} \\ &= \frac{1}{3} \sum_{k=1}^{n} O(k \log k) = O\left(n^2 \log n\right) \end{aligned} \tag{16}$$

The merging phase processes each point exactly once, requiring $O(n)$ candidate searches per point:

$$T_{\text{merging}} = O(n^2) \tag{17}$$

The total time complexity is:

$$T_{\text{total}} = T_{\text{decomposition}} + T_{\text{merging}} = O(n^2 \log n) + O(n^2) = O(n^2 \log n) \tag{18}$$

Space Complexity. The algorithm stores layer information and candidate lists:

$$\begin{aligned} S_{\text{total}} &= O(n) \text{ (layer storage)} + O(n) \text{ (path storage)} \\ &\quad + O(1) \text{ (candidate storage)} = O(n) \end{aligned} \tag{19}$$

4.2 Expected-Case Analysis

Establishing a theoretical upper bound for this TSP algorithm's approximation ratio proves intractable due to convex-hull/greedy-insertion architecture preventing conventional worst-case analysis. Thus, this document proposes experimental validation to achieve practical performance bounds.

5 EXPERIMENTS

5.1 Held-Karp Algorithm Scalability

Held-Karp algorithm faces challenge with increasing problem size due to its exponential growth rate. Figure 1 demonstrates its memory usage growth rate as the number of nodes increases.

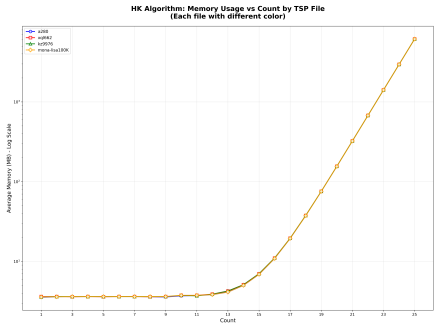

Figure 1: Memory growth of Held-Karp algorithm as problem size increases

Our experiments show that while execution time remained manageable at 25 elements, (a280: 43.86s, xql662: 39.93s, kz9976: 41.25s, mona-lisa100K: 36.52s), memory consumption became the limiting factor by reaching approximately 6.15 GB for all datasets, making further experiment impossible on our hardware. This was due to the theoretical $O(n \cdot 2^n)$ space complexity of the algorithm.

5.2 Algorithm Performance Comparison

Figures 2, 3, and 4 show comparison between three implemented algorithms (Christofides heuristic(CH), MST2, and myown) across the test datasets.

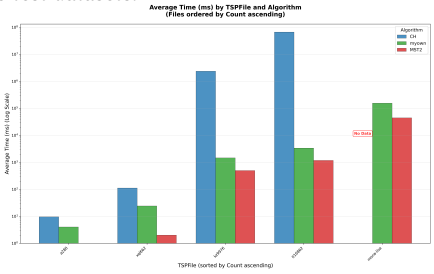

Figure 2: Execution time comparison (log scale).

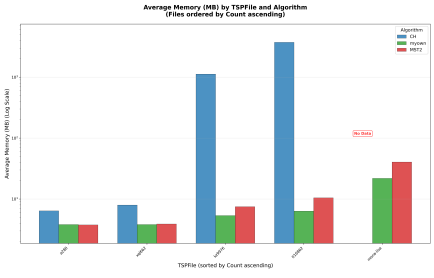

Figure 3: Memory usage comparison (log scale).

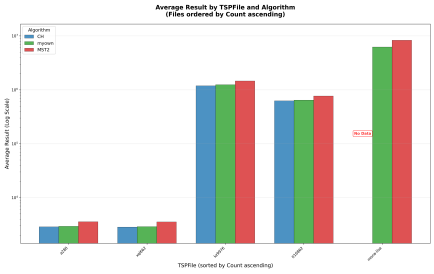

Figure 4: Solution quality comparison (log scale).

The graph reveal clear trade-offs between algorithms. The MST2 demonstrates superior efficiency in both time and memory usage, while CH shows significantly highest costs for larger instances. Our proposed algorithm (myown) achieves a balance between computational efficiency and solution quality.

5.3 Algorithm Performance Summary

Table 1 presents a comprehensive comparison of all implemented algorithms across the mandatory datasets, ordered by instance size.

TSP File	Count	MST2	CH	myown	Optimal
a280.tsp	280	3572 (1.39)	2879 (1.12)	2946 (1.14)	2579
xql662.tsp	662	3547 (1.41)	2847 (1.13)	2897 (1.15)	2513
kz9976.tsp	9976	1457293 (1.37)	1186500 (1.12)	1243247 (1.17)	1061882
it16862.tsp	16862	766671 (1.38)	625164 (1.12)	645942 (1.16)	557315
mona-lisa100K.tsp	100000	8322297 (1.45)	-	6225693 (1.08)	5757084

Table 1: Performance comparison with approximation ratios (algorithm result / optimal result).

While Held-Karp algorithm provides exact solutions, its exponential time and space complexity ($O(n^2 \cdot 2^n)$) makes it impractical for instances beyond 25 nodes. Among practical algorithms, CH consistently achieves the best approximation ratios, but requires significant computational resources, with execution times reaching 6.6×10^7 ms for large instances.

The MST-based 2-approximation offers the best scalability with $O(n^2)$ time and $O(n)$ space complexity. However, it produces the highest approximation ratios (1.37-1.45).

Our proposed algorithm (myown) achieves a favorable balance, providing better approximation quality than MST2 while maintaining reasonable computational efficiency.

5.4 Proposed Algorithm Bound Analysis

Figure 5 presents the approximation ratio analysis of our proposed algorithm with various TSP instances from comprehensive dataset.

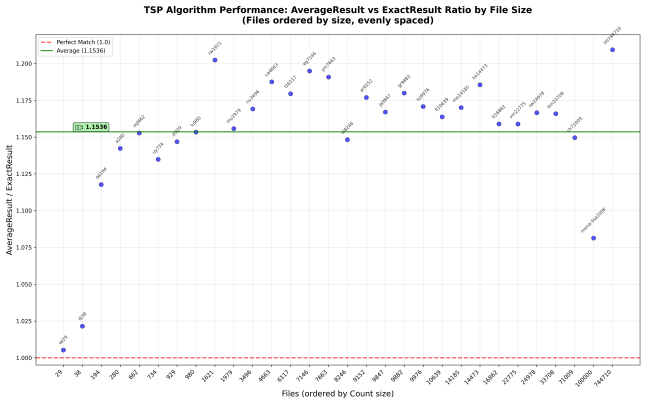

Figure 5: Approximation ratio performance of the proposed algorithm with an average ratio of 1.1536.

While theoretical upper bound verification remains challenging, our empirical analysis demonstrates that the proposed algorithm achieves remarkably consistent performance. The average approximation ratio of 1.1536 across all tested instances, with all instances falling under 1.25. This result suggests that the practical bound for our algorithm is substantially low, making it highly suitable for real-world applications where near-optimal solutions are required with computational efficiency.

6 Conclusion

This project conducted a comprehensive evaluation of TSP algorithms, highlighting trade-offs between computational cost and solution quality. While the Held-Karp algorithm provides exact solutions, its exponential time complexity of $O(n^2 \cdot 2^n)$ makes it impractical for instances beyond 25 nodes.

Accordingly, algorithm selection depends on application requirements: MST2 are suitable for time-critical tasks, Held-Karp is ideal for small-scale exact solutions, and Christofides when approximation quality is critical despite higher resource usage.

Our proposed method creates balance. Achieving near-Christofides solution quality with MST-like scalability by employing geometric decomposition and greedy merging strategies.

References

[1] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. 2009. *Introduction to Algorithms* (3rd ed.). MIT Press, Cambridge, MA. 1111–1117 pages.

[2] Michael Held and Richard M. Karp. 1962. A dynamic programming approach to sequencing problems. *J. Soc. Indust. Appl. Math.* 10, 1 (1962), 196–210.

[3] Vladimir Kolmogorov. 2009. Blossom V: A new implementation of a minimum cost perfect matching algorithm. *Mathematical Programming Computation* 1, 1 (2009), 43–67. doi:10.1007/s12532-009-0002-8

[4] skytre0. 2025. CSE33101: Git Repository. GitHub Repository. https://github.com/skytre0/CSE33101.git Accessed: 2025-06-05.